Group 19: Clinic Appointment System
Phase 2

Table of contents

Contents

Group Information

	Name and Surname	Student number	Contact Information	Group Involvement
1	Ntokozo Maphumulo (Group leader)	43224105	43224105@mynwu.ac.za 0732755567	Contributed
2	Lehlohonolo Matlala	45094454	45094454@mynwu.ac.za 0646879516	Contributed
3	Nyeleti Shivuri	45066132	45066132@mynwu.ac.za 0785849218	Contributed
4	Msizi Mazibuko	41865073	41865073@mynwu.ac.za 0729481489	Contributed
5	Thabiso Tharaka	45225524	45225524@mynwu.ac.za 0680102622	Contributed
6	Phokuhle Shezi	45225524	45225524@mynwu.ac.za 0727001576	Contributed
7	Edzani Maisha	38041693	38041693@mynwu.ac.za 0818246874	Contributed
8	Imameleng Makutoane	42510716	42510716@mynwu.ac.za 0639419091	Contributed
9	Maluleke Kurhula Success	48277444	48277444@mynwu.ac.za 0640708649	Contributed

1.Conceptual Design

1.1 Business rules

The following rules for database design are drawn from system operations and expressed in formal logic.

Appointment Booking Rules

· A patient can book multiple appointments, but each appointment must belong to exactly one patient.

· Only one staff member (nurse or doctor) may be allocated to each appointment.

· Every appointment needs to have a valid time and date.

· A patient has the option to reschedule or cancel an appointment.

Time Allocation Rules

· A time slot becomes available when an appointment is cancelled.

· Every appointment has a single time slot assigned to it.

· A daily limit of 50 appointments can be accommodated by the clinic.

Patient Management Rules

· A student patient must authenticate using their student ID before booking or accessing records

· It is not permitted to have duplicate patient records.

· Every patient has access to their personal records.

Staff Management Rules

· Every employee needs to have a role.

· Every doctor needs to have a license number and a specialization.

· Every employee needs to have specific working hours.

Walk-in Patient Rules

· Walk-in patients are allowed without prior booking.

· Walk-in patients must be given an available time slot.

Queue and Check-in Rules

· A check-in record is required for every appointment.

· Every consultation needs to have a start and end time.

System Integrity Rules

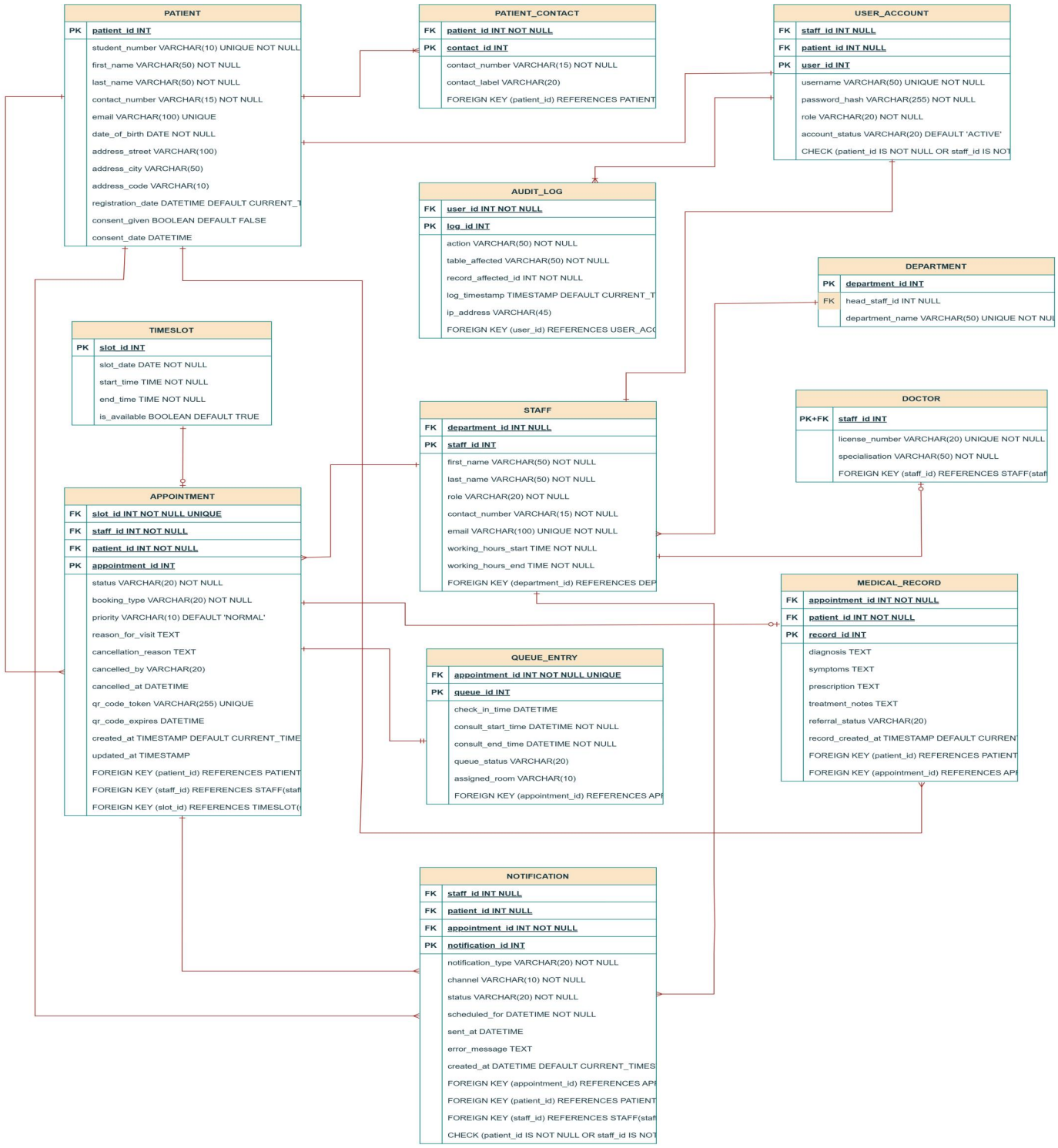
· All appointments must be validated before confirmation.

- Both the patient and the appointment must be connected to the medical records.

Notification Rule

- A reminder must be sent out 24 hours prior to each scheduled appointment.

1.2 Entity Relationship (ER) diagram



1.3 Relationships Between Our Tables

PATIENT → PATIENT_CONTACT

One-to-Many (1:M)

An individual patient may register several contact numbers, yet each contact entry corresponds to a single patient only. The foreign key attribute patient_id is stored within the PATIENT_CONTACT table.

Participation:

PATIENT_CONTACT depends entirely on PATIENT (a contact record cannot exist without a patient)

PATIENT may exist independently of PATIENT_CONTACT (a patient need not provide multiple contacts)

PATIENT → USER_ACCOUNT

One-to-One (1:1)

A patient may be linked to at most one portal account. The connection is established through the patient_id foreign key situated in the USER_ACCOUNT table.

Participation:

USER_ACCOUNT requires a valid PATIENT reference when associated with a patient

PATIENT participation is optional (account creation occurs separately from registration)

USER_ACCOUNT → AUDIT_LOG

One-to-Many (1:M)

A single user account may produce numerous audit entries as actions are performed within the system. Each log entry references exactly one user account via the user_id foreign key in the AUDIT_LOG table.

Participation:

AUDIT_LOG depends completely on USER_ACCOUNT (every log requires an actor)

USER_ACCOUNT may exist without logs (newly created accounts have no history)

DEPARTMENT → STAFF

One-to-Many (1:M)

A department may contain several staff members, whereas each staff member belongs to at most one department. The department_id foreign key resides in the STAFF table.

Participation:

STAFF association with DEPARTMENT is optional (personnel may be unassigned)

DEPARTMENT may exist without STAFF (a newly formed department may be empty)

STAFF → DEPARTMENT (heads)

One-to-Zero-or-One (1:0:1)

A staff member may be designated as the head of a department, holding this position for zero or one department. The head_staff_id foreign key is placed within the DEPARTMENT table.

Participation:

STAFF participation is optional (the majority of staff do not head departments)

DEPARTMENT participation is optional (leadership positions may remain vacant)

STAFF → DOCTOR

One-to-Zero-or-One (1:0:1)

A staff member may optionally hold the role of doctor. The DOCTOR table utilizes staff_id as both its primary identifier and foreign reference.

Participation:

DOCTOR requires an associated STAFF record (every doctor is a staff member)

STAFF may exist without a DOCTOR record (nurses and administrators are not doctors)

STAFF → USER_ACCOUNT

One-to-One (1:1)

A staff member may possess at most one system account. The staff_id foreign key is maintained within the USER_ACCOUNT table.

Participation:

STAFF participation is optional (not all personnel require system access)

USER_ACCOUNT participation is optional (accounts may alternatively belong to patients)

STAFF → APPOINTMENT

One-to-Many (1:M)

A staff member may be scheduled for numerous appointments, while each appointment is assigned to precisely one provider. The staff_id foreign key is stored in the APPOINTMENT table.

Participation:

APPOINTMENT requires a STAFF assignment (every booking needs a provider)

STAFF may have no APPOINTMENT records (newly hired personnel)

TIMESLOT → APPOINTMENT

One-to-One (1:1)

Each time slot accommodates at most one appointment, and every appointment occupies exactly one time slot. The slot_id foreign key in APPOINTMENT carries a UNIQUE constraint.

Participation:

APPOINTMENT depends on TIMESLOT (a booking must have a scheduled time)

TIMESLOT may remain unoccupied (available slots await booking)

APPOINTMENT → QUEUE_ENTRY

One-to-One Mandatory (1:1)

Every appointment generates exactly one queue entry upon patient arrival. The appointment_id foreign key in QUEUE_ENTRY includes a UNIQUE constraint.

Participation:

QUEUE_ENTRY cannot exist without APPOINTMENT (weak entity)

APPOINTMENT must have a corresponding QUEUE_ENTRY (check-in is required)

APPOINTMENT → MEDICAL_RECORD

One-to-Zero-or-One (1:0:1)

An appointment may result in the creation of a medical record, limited to a single record per visit. The appointment_id foreign key is placed in MEDICAL_RECORD.

Participation:

MEDICAL_RECORD requires an APPOINTMENT (documentation tied to specific visit)

APPOINTMENT may lack a MEDICAL_RECORD (missed appointments generate no documentation)

PATIENT → APPOINTMENT

One-to-Many (1:M)

A patient may schedule multiple appointments throughout their history with the clinic. The patient_id foreign key resides in the APPOINTMENT table.

Participation:

APPOINTMENT requires a PATIENT (every booking needs a patient)

PATIENT may have no APPOINTMENT records (newly registered individuals)

PATIENT → MEDICAL_RECORD

One-to-Many (1:M)

A patient may accumulate numerous medical records spanning multiple visits. The patient_id foreign key is stored in the MEDICAL_RECORD table.

Participation:

MEDICAL_RECORD requires a PATIENT (documentation belongs to an individual)

PATIENT may have no MEDICAL_RECORD entries (no prior visits)

APPOINTMENT → NOTIFICATION

One-to-Many (1:M)

An appointment may trigger several notifications throughout its lifecycle. The appointment_id foreign key is placed in the NOTIFICATION table.

Participation:

NOTIFICATION requires an APPOINTMENT (messages tied to specific booking)

APPOINTMENT may have no NOTIFICATION records (messages generated as required)

PATIENT → NOTIFICATION

One-to-Many (1:M)

A patient may receive numerous notifications from the system. The patient_id foreign key resides in the NOTIFICATION table.

Participation:

NOTIFICATION may reference a PATIENT or alternatively a STAFF member

PATIENT may have no NOTIFICATION entries

STAFF → NOTIFICATION

One-to-Many (1:M)

A staff member may receive multiple notifications regarding schedule updates and appointments. The staff_id foreign key is stored in the NOTIFICATION table.

Participation:

NOTIFICATION may reference a STAFF member or alternatively a PATIENT

STAFF may have no NOTIFICATION entries

1.4 Derived Attributes

PATIENT Table

The stored date_of_birth value enables dynamic calculation of the patient's age whenever required. Computing age at query time eliminates the maintenance burden of updating a static age column annually.

QUEUE_ENTRY Table

The duration a patient spends waiting is determined by calculating the interval between consult_start_time and check_in_time. This derived measurement supports the clinic's objective of maintaining wait times under fifteen minutes without requiring additional storage.

APPOINTMENT Table

Certain appointment states may be inferred from contextual data rather than stored explicitly. When the scheduled time passes without an associated check-in record, the appointment can be interpreted as a no-show without necessitating a dedicated status update.

1.5 Normalization Description

First Normal Form (1NF)

Every attribute throughout the schema holds atomic values exclusively. The PATIENT table decomposes address information into separate columns for street, city, and postal code rather than storing concatenated text. The PATIENT_CONTACT entity addresses the multi-valued nature of telephone numbers by allocating a dedicated row for each contact entry. No repeating groups appear within any table structure.

Second Normal Form (2NF)

All tables within the design utilize single-column surrogate primary keys. This approach eliminates any possibility of partial dependency since composite keys do not exist within the schema. Where candidate keys appear, such as student_number in PATIENT, all non-key attributes remain fully dependent on the primary key alone.

Third Normal Form (3NF)

The schema contains no transitive dependencies. The DOCTOR table demonstrates this principle by relocating license_number and specialisation away from the STAFF table. These attributes depend

upon the condition of holding a doctor role rather than upon the staff identifier itself. Isolating these fields in a subtype table prevents update anomalies and removes the necessity for NULL entries among non-doctor personnel. Similarly, the APPOINTMENT table references provider details through foreign keys rather than duplicating attributes such as physician names, ensuring that modifications to staff information occur in a single location.

2.Logical Design